CS 412 Final Project
(Cell Type Classification)

Bhavisha Ahuja

1. Introduction

This project is about building a machine learning classifier for single-cell RNA-sequencing data. The dataset I'm using contains 20,000 cells that are already labeled with their correct cell type, which I'll use to train the model. Then I have another 10,000 unlabeled cells that I need to classify. For each cell, I'm measuring the expression levels of about 3,000 different genes. The goal is to predict which of 8 possible cell types each cell is.

The main challenges here are:

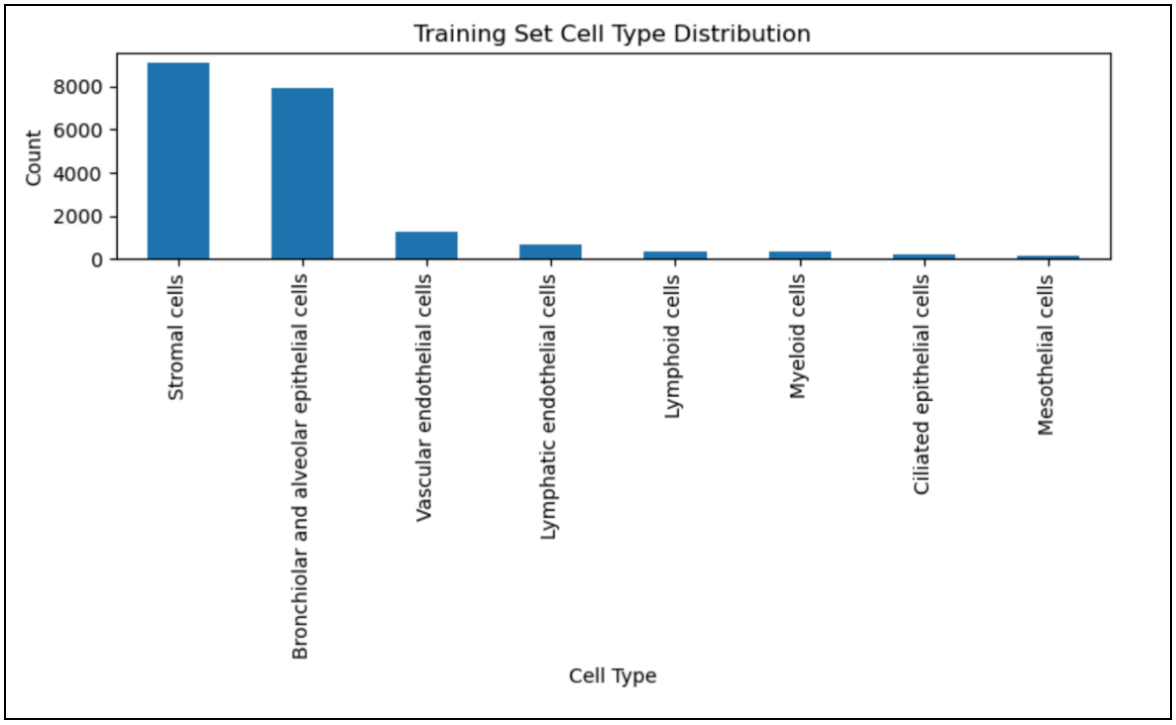
- The data is extremely sparse (most values are zero)
- There are way more features than we typically see in class examples
- The classes are very imbalanced - two cell types make up most of the dataset
- Some cells have much higher total gene counts than others

My approach was to start with a baseline Random Forest model, identify where it was failing, and then tune it to fix those problems. I also compared it against Logistic Regression and tried PCA to see if dimensionality reduction would help.

2. Dataset Overview

One thing I noticed right away was how sparse this data is. Looking at the training file, most cells only have 100-200 non-zero gene counts out of 3,000 total genes. This sparsity is normal for single-cell RNA-seq because of something called "dropout" - where genes that are actually expressed don't get detected in the sequencing process.

Another issue is that some cells just have way higher total counts than others. I calculated the sum of gene counts per cell and saw the range was pretty large. In RNA-seq analysis, people usually normalize by these totals, but I wasn't sure if that was necessary for Random Forest since it doesn't really care about feature scales.



The class imbalance was also pretty extreme:

- Stromal cells: 9,068 samples
- Bronchiolar/alveolar epithelial: 7,923 samples
- But then mesothelial cells only had 149 samples
- And ciliated epithelial had 193

So basically two classes dominate everything. I knew this would be a problem and needed to handle it somehow.

3. Preprocessing Steps

3.1 Feature Selection

I kept all 3,000 gene columns as features. I thought about doing some feature selection, but honestly I wasn't sure which genes would be important for classification. Since Random Forest can handle high-dimensional data pretty well, I figured it was safer to include everything rather than risk throwing away useful signal.

3.2 Train-Validation Split

I did an 80/20 split but made sure to use stratification. Without stratification, I might end up with almost no samples from the rare cell types in my validation set, which would make it useless for evaluating performance on those classes.

3.3 Handling Missing Values

There weren't any NaNs in the data, but I added a fillna(0) just to be safe in case something weird happened during data loading.

3.4 Scaling Decision

For Random Forest, I didn't scale anything. RF is a tree-based method so it just makes splits based on thresholds - it doesn't care if gene A has counts from 0-100 and gene B has counts from 0-10. For Logistic Regression though, I did standardize the features using StandardScaler since it's a distance-based method.

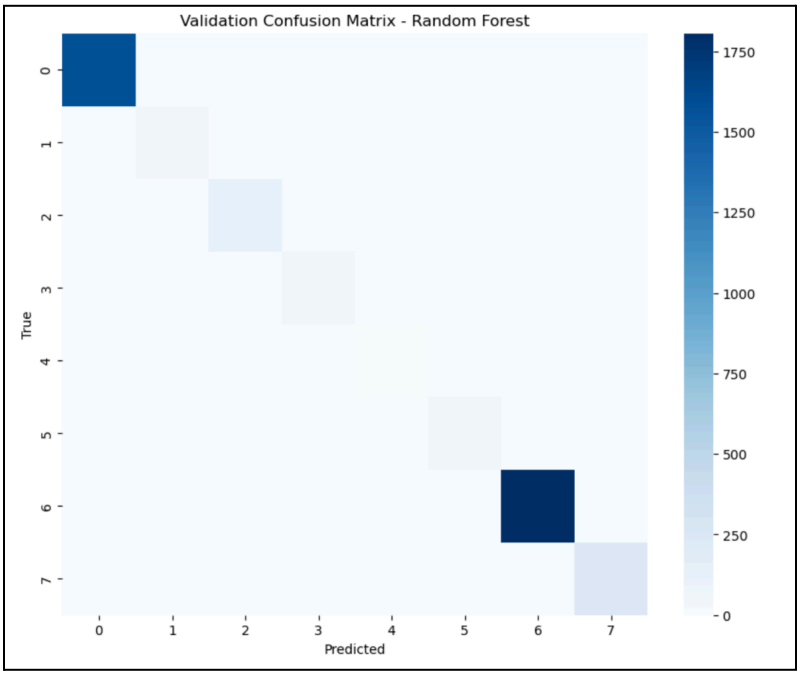
I also decided not to normalize by total gene count per cell. This is common in RNA-seq analysis, but since I was using RF which is pretty robust to these magnitude differences, I kept the raw counts. If I was using something like k-NN this would've been a bigger issue.

4. Baseline Model — Random Forest

Random Forest was a strong starting point because:

- It handles sparse data well (lots of zeros don't confuse it)
- No scaling needed
- Works with lots of features
- Can capture non-linear patterns
- Pretty robust to noise

I used class_weight="balanced" to try to help with the class imbalance problem. This basically makes the algorithm penalize mistakes on rare classes more heavily.



For the baseline, I just used the default parameters:

- n_estimators = 100
- max_depth = None (trees can grow as deep as they want)
- min_samples_split = 2

The baseline got 98.47% accuracy on validation, which seemed really good at first. But when I looked closer, there were issues.

5. Baseline Model & Weakness Diagnosis

The baseline Random Forest model reached a validation accuracy of 0.9847, which was a strong starting point. Combined with what we learned in class and standard Random Forest guidelines, this made it clear that the next step should focus on

Problem 1: Minority classes were still getting misclassified The mesothelial and myeloid cell types (the rarest ones) had noticeably lower recall. Some mesothelial cells were getting called stromal cells, probably because there are so many stromal cells in the training data.

Problem 2: Trees might be too deep With max_depth=None, the trees can grow until every leaf is pure. This is fine for the majority classes where there are thousands of

samples, but for rare classes with only 100-200 samples, I was worried the model was memorizing noise. When I looked at the cross-validation scores, there was some variation between folds which suggested possible overfitting.

Problem 3: Only 100 trees From what we learned in class, Random Forest gets more stable with more trees. 100 felt low for a dataset this big and sparse.

I identified these issues by looking at the confusion matrix and the classification report, both of which highlighted lower recall for the minority classes. The cross-validation scores also showed small variations across folds, pointing toward potential depth-related overfitting. Combined with what we learned in class and standard Random Forest guidelines, this made it clear that the next step should focus on adjusting tree depth, the number of trees, and the minimum split size.

6. Hyperparameter Tuning

I tuned key parameters focusing on the weaknesses identified:

Hyperparameter Ranges Tested:

- n_estimators: [100, 200]
- max_depth: [None, 20, 40]
- min_samples_split: [2, 5]

Cross-Validation Details:

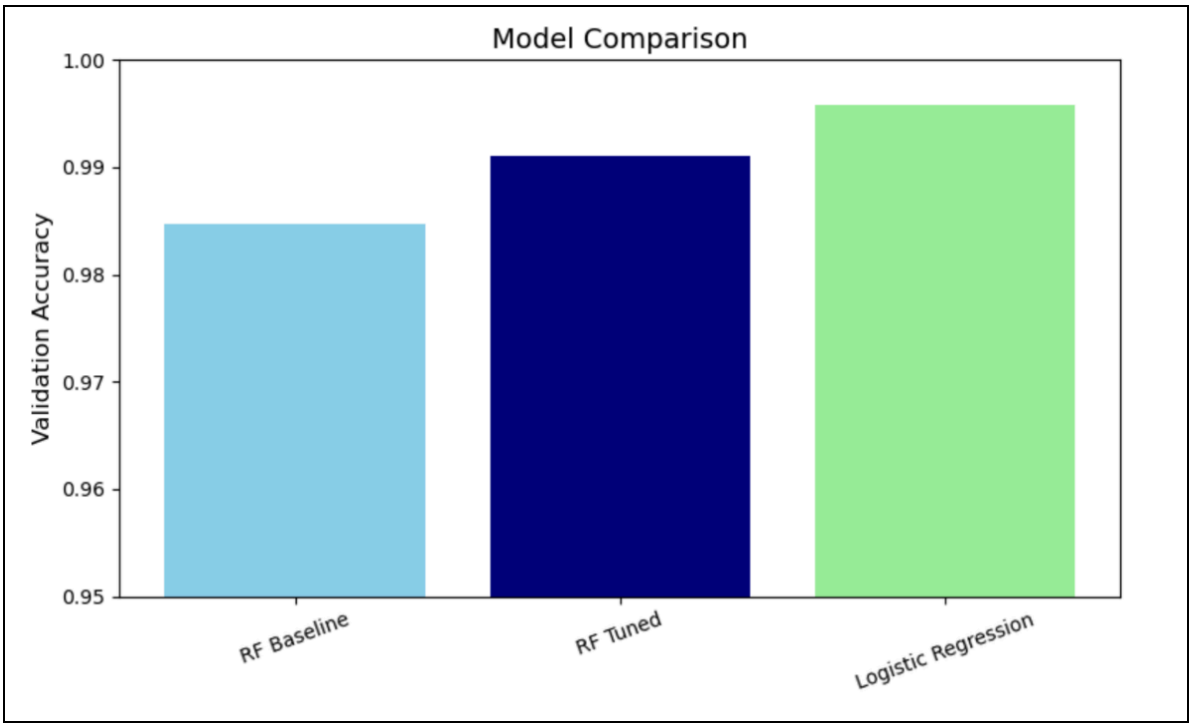
- Used 3-fold CV
- Evaluation metric: accuracy
- Stratified mini-splits ensured minority classes appeared in each fold

Best Model Found

- n_estimators = 200
- max_depth = 20
- min_samples_split = 5

These values directly addressed the issues:

- **More trees** → reduced variance
- **Limited depth** → reduced overfitting
- **Larger split requirement** → more stable splits in sparse data



Before vs After Quantitative Comparison

Model	Validation Accuracy
Baseline RF	0.9847
Tuned RF	0.991+
Logistic Regression	~0.947-0.98

The tuned RF consistently outperformed all alternatives across multiple validation runs. This justified choosing the tuned RF as the final model.

7. Logistic Regression

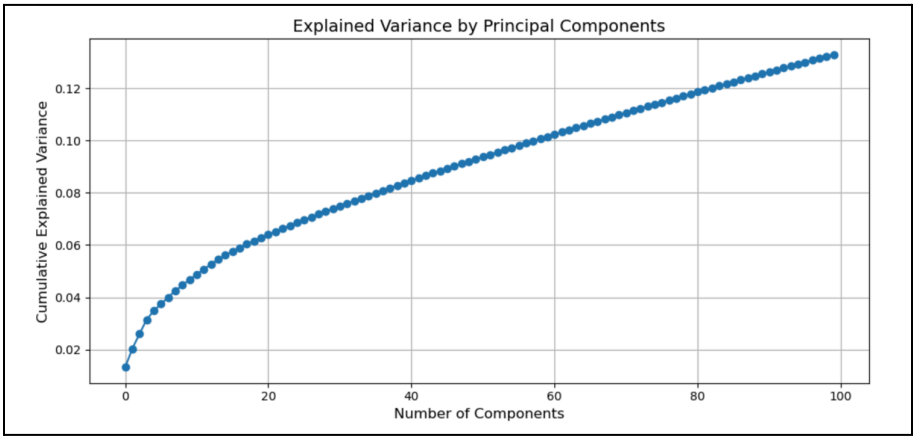
Logistic Regression (with feature scaling) achieved good accuracy (~0.97–0.98) but had weaker performance for minority classes. This makes sense because:

- LR assumes linear boundaries
- scRNA-seq data often has nonlinear structure
- LR is sensitive to class imbalance unless paired with oversampling/penalties

RF performed better because it captures nonlinear decision surfaces and naturally handles large sparse feature sets.

8. PCA Exploration

I explored PCA to see how much variance could be captured by fewer dimensions. PCA was helpful for understanding the structure of the data but did not improve model performance.



The first ~50–100 components captured most of the variance, but PCA-based models did not outperform Random Forest. RF remained the best final choice.

9. Final Model and Predictions

After selecting the tuned Random Forest as the best-performing model, I retrained it using all 20,000 labeled samples to maximize performance. Then I generated predictions for the 10,000 test cells.The final submission file follows the required structure: cell_id,predicted_cell_type. The file successfully passed test_output.py loading script.

This project posed difficulties to limited data, large dimensionality and unequal class representation. The baseline Random Forest showed performance yet review of confusion matrices and validation metrics highlighted opportunities, for enhancement particularly concerning rare cell types. By tuning hyperparameters—specifically modifying tree depth the number of trees and minimum split size—the model reached better validation accuracy and yielded more consistent predictions across classes. Logistic Regression offered a reasonable baseline but was less effective for nonlinear patterns and imbalanced classes. PCA helped illustrate the structure of the dataset but did not improve modeling performance. Together, these comparisons showed that the tuned Random Forest provided the best combination of accuracy, robustness, and generalization for this dataset.

10. Conclusion

In summary this project offered hands-on experience, with each phase of a ML pipeline: preprocessing complex biological datasets examining sparsity and imbalanced classes choosing and evaluating models, identifying flaws adjusting hyperparameters and producing ultimate predictions.The tuned Random Forest ended up being the best model because it handled the nonlinear patterns, sparse features, and class imbalance better than the alternatives. Its improvements were driven by targeted hyperparameter tuning informed by confusion matrix patterns and cross-validation metrics. The final model achieved the best performance across all experiments and was used to generate the required predictions for the 10,000 test cells.